

2교시: Dockerfile 기본 문법

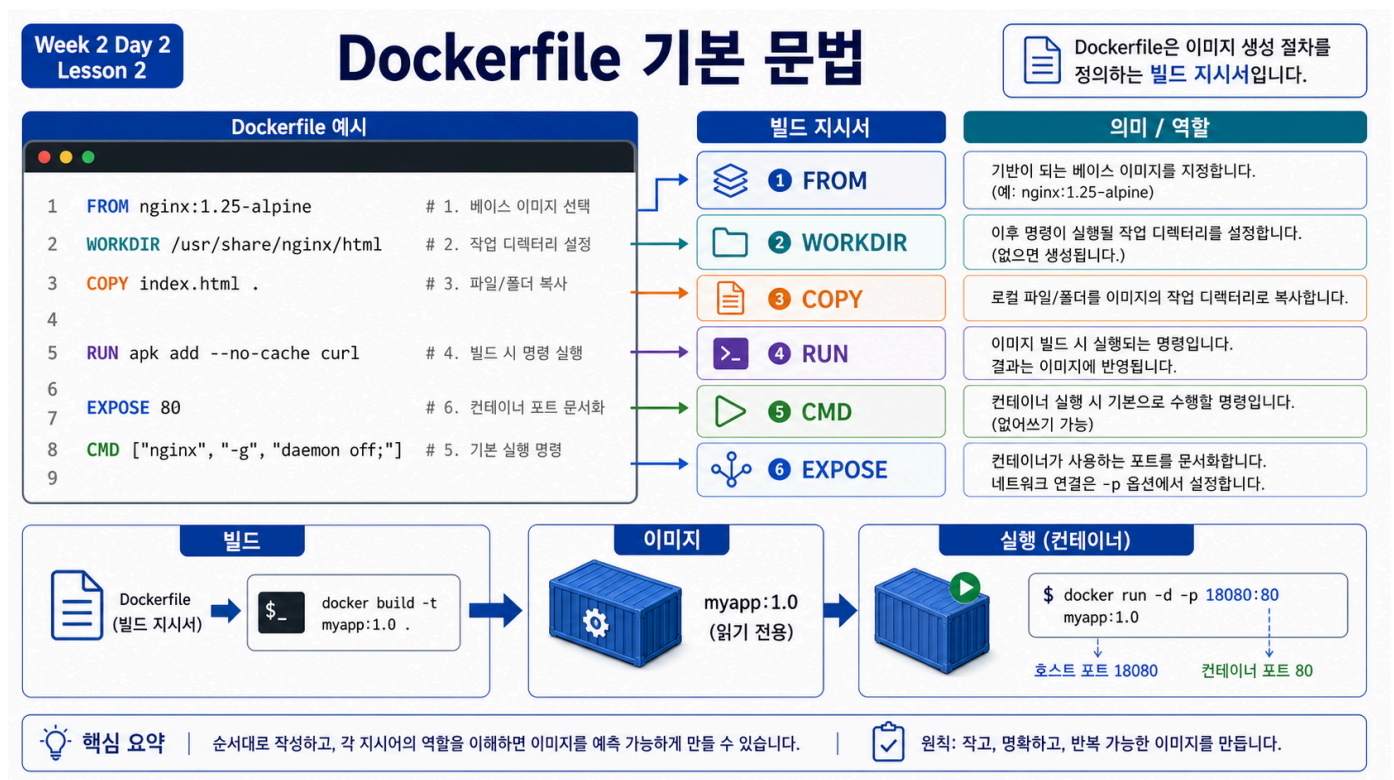
수업 목표

- FROM, WORKDIR, COPY, RUN, CMD, EXPOSE 의 역할을 구분한다.
- build-time instruction과 runtime default를 구분한다.
- Day 2 표준 실습 앱 Dockerfile을 읽고 실행 계약을 설명한다.

50분 흐름

시간	활동	비중	학생 산출
0-8분	Dockerfile이 필요한 이유	설명 15%	실행 조건 list
8-22분	instruction 역할 설명	설명 25%	instruction map
22-38분	실습 Dockerfile 읽기/수정	실행 35%	Dockerfile note
38-45분	build-time/runtime 구분	설명 15%	비교표
45-50분	4교시 build 준비	실행 10%	path 확인

Visual 1: Dockerfile 기본 문법



이 이미지는 Dockerfile instruction을 실행 조건으로 매핑한다. FROM 은 base image, COPY 는 소스 포함, CMD 는 기본 실행 명령, EXPOSE 는 container port 문서화로 읽는다.

실습 Dockerfile

[labs/static-site/Dockerfile:](#)

```
FROM nginx:1.27-alpine
```

```
WORKDIR /usr/share/nginx/html
```

```
COPY index.html styles.css ./
```

```
EXPOSE 80
```

이 Dockerfile은 nginx base image를 사용한다. WORKDIR 는 nginx가 정적 파일을 제공하는 directory로 이동하고, COPY 는 HTML/CSS를 그 위치로 복사한다. EXPOSE 80 은 container 내부에서 80번 port를 사용한다는 문서 역할을 한다. 실제 host port 연결은 docker run -p 18082:80 에서 한다.

Dockerfile을 source code처럼 읽기

Dockerfile은 "실행해 볼 명령 모음"이 아니라 image를 만드는 recipe다. 같은 Dockerfile과 같은 build context, 같은 base image가 있으면 같은 결과에 가까운 image를 만들 수 있다. 반대로 base image tag가 바뀌거나 build context에 다른 파일이 들어가거나 cache가 다르게 작동하면 결과가 달라질 수 있다.

FROM nginx:1.27-alpine 은 수업에서 의도적으로 version을 고정한 예다. nginx:latest 를 쓰면 매번 최신 이미지를 따라갈 수 있지만, 수업 재현성은 약해진다. COPY index.html styles.css ./ 는 context 안의 두 파일만 image에 넣는다. 이 방식은 "필요한 파일만 넣는다"는 운영 기준을 보여준다.

build-time과 runtime

구분	Dockerfile instruction	실행 시점	예시
Build-time	FROM, WORKDIR, COPY, RUN	docker build 중	image filesystem 구성
Runtime default	CMD, ENTRYPOINT	docker run 때	nginx process 시작
Metadata	EXPOSE, LABEL, ENV	build 결과에 기록	port/documentation/config default

이 구분은 장애 분석에도 중요하다. RUN 에서 실패하면 build가 실패한 것이고, CMD 로 시작한 process가 죽으면 container runtime 실패다. 문제 위치가 다르면 봐야 할 evidence도 달라진다.

instruction 판단표

Instruction	역할	자주 하는 오해
FROM	base image 선택	아무 image나 써도 된다고 생각함
WORKDIR	이후 명령의 기준 directory	host의 현재 directory와 같다고 생각함
COPY	build context의 파일을 image에 복사	context 밖 파일도 복사 가능하다고 생각함
RUN	build 중 명령 실행	container 시작 때마다 실행된다고 생각함
CMD	container 기본 실행 명령	build 중 실행된다고 생각함
EXPOSE	container port metadata	host port가 자동으로 열린다고 생각함

실습 질문

- 이 Dockerfile에서 image layer를 실제로 늘리는 instruction은 무엇인가?
- EXPOSE 80 만 쓰고 docker run -p 를 생략하면 browser 접속은 어떻게 되는가?
- COPY . ./ 대신 COPY index.html styles.css ./ 를 쓴 이유는 무엇인가?
- base image를 nginx:latest 로 바꾸면 재현성에 어떤 영향이 있는가?

핵심 유의사항

Dockerfile을 shell script처럼 이해하면 RUN 과 CMD 를 오래 헛갈린다. RUN 은 image를 만드는 중에 실행되고, 그 결과가 image에 남는다. CMD 는 image를 container로 실행할 때의 기본 명령이다. 이 차이를 놓치면 "container를 시작할 때마다 패키지가 설치된다" 같은 잘못된 모델을 갖게 된다.

WORKDIR 는 host terminal의 현재 directory를 바꾸는 명령이 아니다. image 내부 filesystem에서 이후 instruction의 기준 path를 정하는 명령이다. pwd 는 host 위치이고, WORKDIR 는 image 내부 위치다.

COPY 는 build context 안에서만 동작한다. ../secret.txt 를 복사하려고 하면 "왜 상위 폴더 접근이 안 되냐"는 질문이 나온다. 이 제한은 보안과 재현성을 위한 boundary다. Docker daemon에게 전달하지 않은 파일을 Dockerfile이 임의로 가져올 수 없어야 한다.

EXPOSE 는 host port publish가 아니다. EXPOSE 80 은 image metadata에 "이 container는 80번 port를 쓴다"는 정보를 남긴다. 브라우저에서 접속하려면 docker run -p 18082:80 처럼 host port와 container port를 연결해야 한다.

instruction별 자주 놓치는 것

Instruction	학생이 자주 하는 말	바로잡을 핵심
FROM	아무거나 시작점으로 쓰면 된다	base image는 보안, 크기, 재현성의 출발점이다
WORKDIR	내 컴퓨터 폴더가 바뀐다	image/container 내부 기준 경로가 바뀐다
COPY	상대 경로면 어디든 복사된다	build context 안의 파일만 복사된다
RUN	container 실행 때마다 돈다	build 중 한 번 실행되어 결과가 layer에 남는다
CMD	build 중 실행된다	container 시작 시 기본 process를 정한다
EXPOSE	port가 자동으로 열린다	metadata일 뿐 host publish는 -p가 한다

학생 실습 확장

아래 질문은 실제 파일을 망가뜨리지 않고 말로 먼저 예측한 뒤, 시간이 되면 별도 tag로 확인한다.

```
docker build -t paperclip-static-site:syntax-check .
docker history paperclip-static-site:syntax-check
docker inspect paperclip-static-site:syntax-check
```

확인할 항목:

- WorkingDir 가 /usr/share/nginx/html 로 기록되는가?
- ExposedPorts 에 80/tcp 가 보이는가?
- COPY layer의 size가 전체 image 중 얼마나 작은가?
- CMD 는 Dockerfile에 직접 쓰지 않았는데 어디서 온 것인가?

마지막 질문이 중요하다. 현재 Dockerfile에는 CMD 가 없지만 container는 nginx를 실행한다. 그 이유는 base image인 nginx:1.27-alpine 이 이미 CMD ["nginx", "-g", "daemon off;"] 를 갖고 있기 때문이다. 즉 Dockerfile은 항상 base image가 가진 설정 위에 내가 추가한 instruction을 얹는 방식으로 읽어야 한다.

실무에서 챙기면 좋은 습관

습관	이유
base image tag를 명시한다	수업/운영 재현성을 높인다
COPY . ./를 남발하지 않는다	secret, log, 불필요한 파일 유입을 줄인다
port는 EXPOSE와 -p를 함께 설명한다	내부 port와 host port 혼동을 줄인다
build 후 history를 본다	어떤 instruction이 크기를 늘렸는지 확인한다
README에 build/run 명령을 적는다	다른 사람이 같은 결과를 재현할 수 있다

학생 기록 템플릿

Lesson 2 Dockerfile Reading

- FROM:
- WORKDIR:
- COPY:
- EXPOSE:
- build-time instruction:
- runtime default:
- host port publish 명령:
- 내가 헛갈렸던 instruction:

강의 연결 포인트

2교시는 문법 암기가 아니라 4교시 build/run의 사전 분석이다. 학생이 Dockerfile을 읽지 못하면 build output도 읽지 못하고, build output을 읽지 못하면 장애가 났을 때 어느 단계가 문제인지 분류하지 못한다.

따라서 수업 끝에는 Dockerfile 한 줄마다 "이 줄은 언제 실행되는가", "filesystem을 바꾸는가", "runtime 설정인가"를 말하게 한다. 이 세 질문에 답하면 초급 Dockerfile 독해는 충분히 시작된 것이다.

마무리 점검

아래 다섯 문장을 완성하면서 Dockerfile 실행 모델을 확인한다.

FROM은 ____를 정한다.
WORKDIR는 ____ 내부의 기준 경로를 정한다.
COPY는 ____ 안의 파일을 image로 복사한다.
EXPOSE는 host port를 여는 것이 아니라 ____다.
CMD는 build 중 실행되는 것이 아니라 ____ 때 기본 명령으로 쓰인다.

이 점검은 문법 시험이 아니라 실행 모델 확인이다. 문장을 완성하지 못하면 4교시에서 build/run 결과를 따라 치더라도 왜 되는지 설명하기 어렵다.

평가 기준

기준	2점 evidence
instruction 구분	6개 instruction의 역할을 설명했다.
runtime 구분	RUN과 CMD 차이를 설명했다.
port 이해	EXPOSE 80과 -p 18082:80 차이를 설명했다.
path 이해	WORKDIR와 COPY 대상 경로를 설명했다.

공식 근거 링크

- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- Docker Docs: Writing a Dockerfile, <https://docs.docker.com/guides/docker-concepts/building-images/writing-a-dockerfile/>